RELATÓRIO TÉCNICO: PROJETO AURORA SIGER

Curso: Ciência da Computação

Trabalho: Atividade Integradora - Relatório operacional de pré-decolagem

Estudante: Aelton Soares de Menezes

1. Organização e Descrição da Telemetria

O sistema monitora seis vetores de dados essenciais para a viabilidade do voo. Abaixo, a descrição técnica e as métricas de segurança estabelecidas:

Parâmetro	Unidade	Descrição	Faixa de Segurança
Temperatura Interna	°C	Temperatura no cockpit e baia de sistemas.	$15 \leq T \leq 40$
Temperatura Externa	°C	Temperatura na superfície externa do casco.	$-150 \leq T \leq 100$
Integridade Estrutural	Binário	Status de sensores de microfissuras e deformação.	1(Nominal)
Nível de Energia	%	Carga disponível nos sistemas de alimentação.	$\geq 85\%$
Pressão dos Tanques	PSI	Pressão interna do sistema de combustível.	$3000 \leq P \leq 4500$
Status de Módulos	Bool	Verificação de integridade dos módulos críticos.	True (Operacional)

2. Algoritmo de Verificação (Pseudocódigo)

Desenvolvido para garantir uma decisão binária e determinística baseada nos dados coletados.

```
ALGORITMO Verificar_Status_Decolagem
INICIO
    LER temp_int, temp_ext, integridade, energia, pressao, modulos

    DEFINIR pronto = VERDADEIRO

    SE (temp_int < 15 OU temp_int > 40) ENTÃO pronto = FALSO
    SE (temp_ext < -150 OU temp_ext > 100) ENTÃO pronto = FALSO
    SE (integridade != 1) ENTÃO pronto = FALSO
    SE (energia < 85) ENTÃO pronto = FALSO
    SE (pressao < 3000 OU pressao > 4500) ENTÃO pronto = FALSO
    SE (modulos == FALSO) ENTÃO pronto = FALSO

    SE (pronto == VERDADEIRO) ENTÃO
        EXIBIR "STATUS: PRONTO PARA DECOLAR"
    SENÃO
        EXIBIR "STATUS: DECOLAGEM ABORTADA"
FIM
```

3. Script em Python

O código foi implementado em ambiente `.ipynb` (Jupyter Notebook), utilizando a biblioteca `random` para simular as leituras dos sensores em tempo real.

https://github.com/Aelton0/Aurora_Siger)

Declaração das variavies de segurança

[39]
✓ 0s

```
# Configurações de Segurança
SAFE_TEMP_INT = (15, 40)        # Entre 15°C e 40°C
SAFE_TEMP_EXT = (-150, 100)     # Entre -150°C e 100°C
MIN_ENERGY = 85                 # Mínimo 85%
MIN_PRESSURE = 3000             # Mínimo 3000 PSI
MAX_PRESSURE = 4500             # Máximo 4500 PSI
```

[40]
✓ 0s

```
def realizar_telemetria():
    # Simulando a leitura dos sensores
    dados = {
        "temp_int": random.uniform(10, 50),
        "temp_ext": random.uniform(-180, 150),
        "integrity": 1 if random.random() > 0.1 else 0, # 90% de chance de estar ok
        "energy_pct": random.uniform(70, 100), # pct é de porcentagem
        "tank_pressure": random.uniform(2500, 5000),
        "critical_modules": True if random.random() > 0.05 else False
    }
    return dados
```

Iniciando os calculos de autonomia

A formula é: energia_atual = capacidade_total_kwh * (carga_pct / 100)

- Consumo na decolagem 400kWh
- Perda de 50kWh

OBS: Como estou usando funções sera definido depois algumas variaveis como capacidade total, consumo na decolagem e perdas.

[34]
✓

```
def calcular_autonomia(capacidade_total_kwh, carga_pct, consumo_decolagem, perdas):
    # Cálculo conforme a fórmula da atividade
    energia_atual = capacidade_total_kwh * (carga_pct / 100)
    autonomia_pos_decolagem = energia_atual - consumo_decolagem - perdas
    return autonomia_pos_decolagem
```

[41]
0s

```python
def verificar_decolagem(telemetria):
    erros = []

    # 1. Verificar Temperaturas
    if not (SAFE_TEMP_INT[0] <= telemetria["temp_int"] <= SAFE_TEMP_INT[1]):
        erros.append(f"Temperatura Interna Crítica: {telemetria['temp_int']:.2f}°C")

    # 2. Verificar Integridade (0 ou 1)
    if telemetria["integrity"] == 0:
        erros.append("Falha na Integridade Estrutural!")

    # 3. Verificar Energia
    if telemetria["energy_pct"] < MIN_ENERGY:
        erros.append(f"Energia Insuficiente: {telemetria['energy_pct']:.2f}%")

    # 4. Verificar Pressão
    if not (MIN_PRESSURE <= telemetria["tank_pressure"] <= MAX_PRESSURE):
        erros.append(f"Pressão dos Tanques Fora do Limite: {telemetria['tank_pressure']:.2f} PSI")

    # 5. Módulos Críticos
    if not telemetria["critical_modules"]:
        erros.append("Erro em Módulos Críticos do Sistema")

    return erros
```

Inicialização do lançamento

[42]
0s

```python
# --- INICIANDO ---
telemetria_atual = realizar_telemetria()  #Chama a função realizar_telemetria e armazena na variavel telemetria_atual
falhas = verificar_decolagem(telemetria_atual) #Chama a função verificar_decolagem e armazena em falhas

print("=== RELATÓRIO DE TELEMETRIA ===")
for chave, valor in telemetria_atual.items():
    print(f"{chave}: {valor:.2f}" if isinstance(valor, float) else f"{chave}: {valor}")

print("\n=== ANÁLISE ENERGÉTICA ===")
# Só para lembrar a ordem dos argumentos da função calcular_autonomia() (capacidade_total_kwh, carga_pct, consumo_decolagem, perdas)
# Exemplo: 5000kWh total, consumo de 400kWh na subida, perda de 50kWh
autonomia = calcular_autonomia(5000, telemetria_atual["energy_pct"], 400, 50)
print(f"Autonomia estimada após decolagem: {autonomia:.2f} kWh")

print("\n" + "="*30)
if not falhas:
    print("STATUS: PRONTO PARA DECOLAR")
else:
    print("STATUS: DECOLAGEM ABORTADA")
    print("MOTIVOS:")
    for erro in falhas:
        print(f"- {erro}")
```

```
=== RELATÓRIO DE TELEMETRIA ===
temp_int: 37.20
temp_ext: -153.52
integrity: 1
energy_pct: 91.15
tank_pressure: 4923.16
critical_modules: True

=== ANÁLISE ENERGÉTICA ===
Autonomia estimada após decolagem: 4107.56 kWh

==============================
STATUS: DECOLAGEM ABORTADA
MOTIVOS:
- Pressão dos Tanques Fora do Limite: 4923.16 PSI
```

4. Análise Energética

A autonomia é calculada com base no balanço energético inicial e as cargas de decolagem. (que eu defini arbitrariamente)

```
# Só para lembrar a ordem dos argumentos da função calcular_autonomia()
(capacidade_total_kwh, carga_pct, consumo_decolagem, perdas)
# Exemplo: 5000kWh total, consumo de 400kWh na subida, perda de 50kWh
autonomia = calcular_autonomia(5000, telemetria_atual["energy_pct"], 400, 50)
```

Parâmetros de Cálculo:

- **Capacidade Total (C):** 5000 kWh
- **Carga Atual (L):** 90% -> Random do energy_pct
- **Consumo na Decolagem (D):** 400 kWh
- **Perdas (P):** 50 kWh

Fórmulas Utilizadas:

1. **Energia Inicial (E.I):** C x L → 5000 x 0.90 = 4500 kWh
2. **Energia Residual (E.R):** E.I - D - P → 4500 - 400 - 50 = 4050 kWh

Resultado: A autonomia pós-decolagem é de **4050 kWh**.

5. Análise Assistida por IA

Utilizando modelos de inteligência artificial para análise preditiva dos dados:

- **Classificação:** Os dados foram rotulados como "Estado Nominal" e "Estado de Risco".
- **Anomalias:** Identificação de flutuações rápidas na pressão dos tanques que, embora dentro da faixa, podem indicar instabilidade futura.
- **Sugestão de Risco:** Risco de fadiga térmica em cenários de temperatura externa extrema persistente.

Os dados fornecidos para a IA:

```
=== RELATÓRIO DE TELEMETRIA ===
temp_int: 10.14
temp_ext: 143.67
integrity: 1
energy_pct: 86.28
tank_pressure: 4592.25
critical_modules: True

=== ANÁLISE ENERGÉTICA ===
Autonomia estimada após decolagem: 3863.80 kWh

==============================
STATUS: DECOLAGEM ABORTADA
MOTIVOS:
- Temperatura Interna Crítica: 10.14°C
- Pressão dos Tanques Fora do Limite: 4592.25 PSI
```

Resposta da IA:

Abaixo, apresento a análise diagnóstica realizada sobre os dados coletados na última simulação do sistema **Aurora Siger**:

A. Classificação dos Dados

Com base no relatório de telemetria, a IA classificou os dados nos seguintes níveis de severidade:

- **Nominal (Operacional):** * **Integridade Estrutural (1):** Status íntegro.
    - **Módulos Críticos (True):** Sistemas de controle respondendo adequadamente.
- **Alerta (Monitoramento Necessário):**

    - **Nível de Energia (86.28%):** Embora acima do limite de 85%, a margem de segurança é baixa (apenas 1.28% acima do mínimo), exigindo gestão eficiente durante a subida.
    - **Temperatura Externa (143.67°C):** Valor elevado que demanda verificação dos sistemas de resfriamento do casco.
- **Crítico (Falha Ativa):**
    - **Temperatura Interna (10.14°C):** Abaixo do limite de conforto e funcionalidade térmica (15°C).
    - **Pressão dos Tanques (4592.25 PSI):** Acima do limite de segurança estrutural de 4500 PSI.

B. Identificação de Possíveis Anomalias

A IA identificou duas correlações perigosas nos dados:

1. **Anomalia de Pressão Positiva:** O valor de **4592.25 PSI** indica uma sobrepressão. Isso pode ser causado por uma falha na válvula de alívio ou por uma reação química não planejada no propelente.
2. **Hipotermia de Sistemas Internos:** A temperatura de **10.14°C** no interior sugere uma falha no sistema de suporte à vida ou no aquecimento dos componentes eletrônicos, o que pode causar falha por condensação ou congelamento de circuitos.

C. Sugestões de Risco

1. **Risco de Explosão ou Ruptura:** A sobrepressão nos tanques aumenta exponencialmente o risco de uma explosão durante a ignição, onde a pressão mecânica e o calor externo aumentam drasticamente.
2. **Risco de Falha de Hardware:** Temperaturas internas muito baixas podem levar à falha de baterias e sensores de precisão, resultando em perda de controle (LOC - *Loss of Control*) durante a trajetória de subida.
3. **Veredito de IA:** Recomendação de **Aborto de Missão** imediato para despressurização dos tanques e recalibragem do sistema térmico interno.

6. Reflexão Crítica

Ética e Responsabilidade: O desenvolvimento de software aeroespacial exige o compromisso inegociável com a segurança. Falhas de código podem resultar em perdas irreparáveis de vidas e recursos.

Impacto Social: A exploração espacial democratiza tecnologias (telecomunicações, meteorologia), mas deve ser acessível e voltada ao progresso coletivo da humanidade.

Sustentabilidade Tecnológica: É imperativo planejar o ciclo de vida dos componentes e a redução de detritos espaciais, garantindo que o avanço tecnológico de hoje não bloqueie o acesso orbital das futuras gerações.